```
proxiedForm.name
# 'property access intercepted by proxy'

proxiedForm.phone = '555 9988'
#  'property modify intercepted by proxy'
```

This is useful for Formulaic. The existing implementation extracts all of the fields and creates an intermediate representation of the form. With ECMAScript 6, instead of this intermediate representation your program can use a proxy. In listing 13.5 you can see `Proxy` and `Map` used to create a new version of Formulaic. In order for this program to work, it will need to execute in a runtime that supports proxies. For that reason, this listing throws an error if proxies are not supported.

Listing 13.5. Formulaic using `Proxy`

```
throw new Error 'Proxy required' unless Proxy?

class Formulaic

  constructor: (@root, @selector, @http, @url) ->
    @source = @root.querySelector @selector
    @handler =
      get: (target, property) ->
        target[property]?.value
      set: (target, property, value) =>
        if @valid property then @sync()
    @fields = new Proxy @source, @handler

  valid: (property) ->
    property isnt ''

  addField: (field, value) ->
    throw new Error "Can't append to DOM" unless
@source.appendChild?

    newField = @root.createElement 'input'
    newField.value = value
    @source.appendChild newField

  sync: ->
    throw new Error 'No HTTP specified' unless @http? and @url?

    @http.post @url, JSON.stringify(@source), (response) => #B
      for field, fieldResponse of JSON.parse response
        if field of @source
```